

AI概念

总结：AI就是预测下一个词的LLM，通过最小单元(token)处理信息，在有限的记忆(context)里根据指令(prompt/harness)，借助外部能力(tool/MCP)，自主完成任务(agent/skill)。

▼ LLM大语言模型

- 几乎都是基于transformer架构训练而成；
本质：一个数学函数，只会猜词，预测下一个词（文字接龙）

▼ 参数量

- 本质：脑细胞多少，数字越大，能学会的东西越多，但是更费钱费时

▪

- 超小/边缘：0.5B-3B（手机、IoT设备）
- 小：7B-8B（入门、高性价比）
- 中：14B-32B（均衡、常用）
- 大：70B-100B（高质量）
- 超大：100B+（顶级能力）
- MoE：总参数大但激活少（高性价比）

真实例子：

- 超小：Qwen2.5-0.5B/1.5B/3B、Llama 3.2-1B/3B、Phi-3-mini
- 小：Llama 3.1-8B、Qwen2.5-7B、Mistral-7B
- 中：Qwen2.5-14B/32B、Gemma 2-27B
- 大：Llama 3.1-70B、Qwen2.5-72B
- 超大：Llama 3.1-405B
- MoE：DeepSeek-V3（671B总/37B激活）、Mixtral 8x22B（141B总/39B激活）

▼ token

- 数据处理的最小计量单位，LLM只认识数字。
tokenizer：编码和解码token
1token≈0.75个英文单词；1token≈1-2个汉字
token代表钱也就是成本

▼ context上下文

- context是LLM每次处理任务时接收到的信息总和 → context window: 记忆体能装下的最大token数量

▼ rag(检索增强生成)

- 面对超长文档, rag检索出和问题最匹配的几页摘出来给大模型, 让模型基于这些片段问答问题

▼ prompt

- ▼ user prompt: 用户给的指令或问题
 - 需求越明确详细越好
- system prompt: 开发者预置的, 但用户看不到

▼ Harness(指令束)【prompt的进阶版】

- prompt关注单词对话的输出质量, 而harness关注的时AI的长期工作模式。因此, 内容是完整的指令体系(身份+行为边界+工具使用规则+输出格式+质量标准.....)
- ▼ 深入理解
 - 一整套用于“驾驭”大模型 (LLM) , 使其能稳定、安全、有效完成实际任务的基础设施与工程环境

▼ 核心职责

- - Prompt组织: 动态拼装系统提示词、任务指令和上下文。
 - 工具路由: 决定当前步骤该调用哪个能力。
 - 上下文与记忆管理: 维护历史对话、短期记忆和长期知识库。
 - 任务流控制: 管理串行、并行、分支、循环和重试。
 - 权限与安全约束: 限制敏感操作、防越权、防误调用。
 - 错误恢复: 处理工具失败、自动回退或重试。
- 一句话概括: Harness解决的是“这个Agent系统怎么跑、怎么控、怎么验”的问题

▼ 适用场景

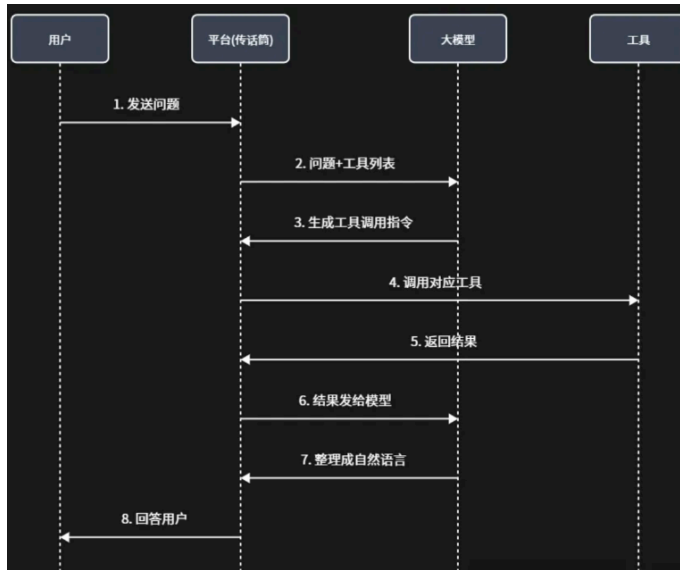
- 复杂任务、Agent系统、工作流

▼ 例子

- Claude code: 一个编程助手、怎样读代码、怎么执行命令、什么时候该问用户、输出格式是什么等等, 这些都是开发者在后台配好的, AI每次启动自动加载。

▼ tool

- 大模型无法回答“今天天气如何”这种问题, tool即为解决方案
- 本质: 感知外部世界的函数(输入参数→执行操作→返回结果)
- ▼ 工作流程



- 平台很重要，大模型不会自己调用工具，而是选择工具再依赖平台调用工具

▼ 痛点：每个平台的工具规范不同，因此产生MCP解决方案

▪

平台	工具规范
ChatGPT	按OpenAI规范接入
Claude	按Anthropic规范接入
Gemini	按Google规范接入

同一个工具，要写三遍代码。

▼ 例子

- 输入（城市+日期）→调用气象接口→赶回天气数据

▼ MCP模型上下文协议

- AI工具的"typeC"接口
同意工具介入标准，写一次代码，所有平台通用
- 含义：遵循给模型看的格式、可以调用的插件，让 AI 能连接外部服务。比如装了 Notion MCP，AI 就能读写你的 Notion；装了 Google Drive MCP，AI 就能访问你的云端文件

▼ agent智能体

- ▼ 能自主干活的系统
 - 能自主规划、调用多个工具、持续工作，知道把任务完成
- 本质：LLM+TOOLS
- ▼ 分类(粗糙)
 - ▼ 平台型agent

- coze\diffy

▼ 通用型agent

- manus\coding agent如claude code codex等

▼ skills

- 给agent的说明书：agent每次干活之前 都能自动读取，不需要用户重复说明

▼ 本质

▼ markdown格式的文档

▪

Skill：执行任务的“技能包”

Skill指的是相对独立、可复用、目标明确的专项能力模块 [m.blog.csdn](#) 。它不负责整体流程，只专注于把某一具体事情做好。一个Skill通常是一个包含 `SKILL.md` 文件的文件夹，里面封装了完成特定任务的步骤、规范和工具 [cloud.tencent](#) 。

常见的Skill形态包括 [m.blog.csdn](#)：

- 读取并解析PDF/Word文档
- 执行特定数据分析或生成图表
- 查询内部数据库或外部API
- 起草并格式化邮件、报告

一句话概括：Skill解决的是“这个Agent具体会做什么”的问题 [blog.csdn](#) 。

▼ 包含内容

▼ 元数据层

▼ 内容：名称+描述

- 作用：让agent知道这个skill是干什么的

▼ 指令层

▼ 内容：目标、执行步骤、判断规则、输出格式、示例

- 作用：告诉agent具体怎么干活

▼ 创建的实操步骤

▪

1. 在 `.cloudskills` 目录下新建一个文件夹（文件夹名称就是Skill名称）
2. 在文件夹里创建 `SKILL.md` 文件（注意：必须大写）
3. 在里面写完整的指令内容
4. Agent在匹配到相关任务时，自动加载执行

🌐 ⚡ 效果：你说一句“我要出门了”，Agent自动调定位、查天气、按预设规则判断要带什么、按你指定的格式输出结果。全自动，不用你说一个字。

▼ prompt v.s. skill

- 1. 一次上传，多次调用。不用每次复制粘贴。
 - 2. 按需加载，不占上下文。Agent 先看 metadata，觉得“这个任务需要用到这个 Skill”，才会去加载具体内容，不会挤占对话窗口。
 - 3. 标准化格式，可以跨平台用。Anthropic 提的这个协议，现在越来越多 Agent 都在支持。
- 所以 Skill 不是替代 Prompt，而是把 Prompt 升级成可复用的能力包。

▼ harness v.s. skill

维度	HARNESS (框架层)	SKILL (能力层)
定位	调度器 / 控制中心 m.blog.csdn	插件 / 专项技能包 m.blog.csdn
类比	操作系统 / 项目经理 m.blog.csdn	应用程序 / 专业员工 m.blog.csdn
核心职责	管流程、状态、安全、容错 m.blog.csdn	管执行、输入输出、业务逻辑 m.blog.csdn
生命周期	贯穿Agent整个运行过程 m.blog.csdn	被调用时激活，完成后返回 m.blog.csdn
关注点	怎么跑 m.blog.csdn	会什么 m.blog.csdn
产出	完整的系统架构、流程、规则	封装好的能力函数 m.blog.csdn

▼ 案例

假设要开发一个自动生成行业研报的Agent，可以这样拆分 [blog.csdn](#)：

- Harness负责宏观调度：
 1. 解析用户意图，明确报告主题、篇幅和输出格式。
 2. 规划执行链路：先搜资讯，再读PDF，再分析数据，最后写总结。
 3. 做权限控制：允许查内部数据库，但禁止直接发送外部邮件。
 4. 做容错：网页抓取失败时自动切换备用数据源，并设置最多重试3次。
 5. 记录完整执行轨迹，方便回溯和优化。
- Skill负责具体执行：
 - `web_search_skill`：联网检索最新行业新闻。
 - `pdf_parser_skill`：提取PDF中的关键段落和表格。
 - `data_analysis_skill`：清洗数据并生成图表。
 - `doc_generator_skill`：按模板输出Markdown或PPT报告。

从这个交互关系看，Harness更像项目经理，负责拆任务、排流程、控风险；Skill更像专业员工，接到明确任务后专注交付结果 [blog.csdn](#)。

▼ MCP与skill如何配合

- MCP 是“连接”，Skill 是“能力”。举个例子：你装了 Notion MCP，AI 就能连接到你的 Notion；但它不知道你的 Notion 里有什么、怎么用。这时候你写一个 Skill，告诉它：“我的 Notion 里有个数据库叫‘选题库’，每次我说‘记录选题’，你就往里面写一条记录”——这样 AI 就知道怎么用 Notion 了

▼ MOE混合专家

- 每次只叫少数专家上场，省算力同时提升能力，但工程更复杂。

- DeepSeek-V3：671B 总参数，每次激活 37B（256 专家，top-8）
- Mixtral 8x7B：47B 总参数，每次激活 13B（8 专家，top-2）
- Mixtral 8x22B：141B 总参数，每次激活 39B
- GPT-4：据传也是 MoE 架构

▼ MOE激活数：每次上场的专家数(top-2即为2个)，数越大越准，但更费算力

■

- DeepSeek-V3 : top-8 (256 专家中选 8 个)
- Mixtral 8x7B : top-2 (8 专家中选 2 个)
- Mixtral 8x22B : top-2 (8 专家中选 2 个)

▼ VL模型(看图说话)

- 能看图/视频又能理解文字，如读图、问图、按文字生成图

▼ 端侧

- 含义：设备本地，此时不需要联网就能运行AI模型

▼ 端侧 v.s. 云端

■

- 云端：数据发到服务器 → 服务器算 → 结果返回（需要网络，有延迟，隐私风险）
- 端侧：数据不出设备 → 本地算 → 本地出结果（离线可用，快，隐私安全）

▼ 端侧的优势

■

优势	说明	典型场景
隐私安全	数据不上传，敏感信息不泄露	医疗诊断、个人日记、企业文档
离线可用	无网络也能用	飞行模式、偏远地区、地下停车场
低延迟	不用等网络往返，毫秒级响应	实时翻译、语音助手、游戏 NPC
省成本	不用付云端 API 费用	高频调用场景（如输入法联想）
个性化	可以针对用户习惯微调	个人助理、学习伴侣

▼ 端侧模型的技术要求

■

- 模型压缩：量化（INT4/8）、剪枝、蒸馏，把大模型"瘦身"
- 推理优化：专用芯片（NPU/神经引擎）、算子融合、内存优化
- 设备要求：
 - 手机：需要 4GB+ RAM，支持 NPU 更好（如骁龙 8 Gen3、天玑 9300、A17 Pro）
 - 电脑：8GB+ RAM，有独显更好
 - IoT：需要专门优化的超小模型（< 500MB）

▼ 端侧模型的商业案例

■

💬 输入法联想

案例：搜狗输入法、讯飞输入法

功能：• 智能纠错

• 下一句预测

• 表情推荐

为什么用端侧：高频调用（每秒多次），云端成本太高，且需要毫秒级响应

🚗 智能汽车

案例：理想汽车、小鹏汽车的车载助手

功能：• 语音控制（“打开天窗”）

• 导航问答（“附近哪有充电桩”）

• 儿童陪伴（讲故事、问答）

为什么用端侧：车库/隧道无网络，安全要求高（不能断网就失效）

🏠 医疗设备

案例：便携式超声诊断仪

功能：• AI 辅助诊断（识别病灶）

• 报告自动生成

为什么用端侧：患者隐私（医疗数据不能上传），偏远地区无网络

🏠 智能家居

案例：小爱音箱、天猫精灵（离线版）

功能：• 语音控制家电（“关灯”）

• 简单问答（天气、闹钟）

为什么用端侧：网络不稳定时也能用，隐私安全（不录音上传）

🎓 教育设备

案例：科大讯飞学习机、网易有道词典笔

功能：• 拍照识题 + 解答

• 口语评测

• 作文批改

为什么用端侧：学生隐私保护，考试场景不能联网

💼 企业办公

案例：Microsoft 365 Copilot（本地模式）

功能：• Word 文档润色

• Excel 数据分析

• 会议纪要生成

为什么用端侧：企业敏感数据不能上云

常见任务 → 直接建议

- 企业知识问答：Qwen2.5-14B / Llama 3.1-8B + RAG，上下文 32k-64k
- 客服短对话：Qwen2.5-7B / Llama 3.1-8B，量化部署
- 代码/数学：DeepSeek-R1 / o1 / Claude 3.5 Sonnet
- 看图相关：GPT-4o / Qwen2-VL / Claude 3.5 Sonnet
- 超长文档：Gemini 1.5 Pro（100万） / Claude 3.5（200K）
- 手机/边缘/端侧：Qwen2.5-0.5B/1.5B / Llama 3.2-1B/3B / Phi-3-mini，量化